CSA1708-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Program 1 : Python Program to Solve 8. Puzzle Problem

Aim

To develop a Python program that solves the 8-Puzzle Problem using a search algorithm (such as Breadth-First Search - BFS) to find the shortest sequence of moves from the initial state to the goal state.

Algorithm

1. Represent the puzzle as a 3×3 matrix.
2. Define the goal state.
3. Locate the empty tile (0).
4. Generate all possible moves (up, down, left, right).
5. Use Breadth-First Search (BFS) to explore possible states.
6. Keep track of visited states to avoid repeated exploration.
7. If the goal state is reached, display the solution path.
8. If all possibilities are exhausted, report that no solution exists.

S PUZZLE.py - C:/Users/jasmi/OneDrive/Pictures/Documents/S PUZZLE.py (3.14.4)

File Edit Format Run Options Window Help

```
from collections import deque

GOAL = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

def get_neighbors(state):
    neighbors = []

    # Find the blank tile (0)
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            new_state[x][y], new_state[nx][ny] = (
                new_state[nx][ny],
                new_state[x][y],
            )

            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors


def bfs(start):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == GOAL:
            return path + [state]

        if state in visited:
            continue
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/S PUZZLE.py =======
Solution Found!

(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

>>>
```

RESULT

Thus, the 8-Puzzle Problem solved successfully using Python.

Program 2: Python Program to Solve 8 Queen Problem

Aim

To develop a Python program to solve the 8-Queen Problem using the Backtracking Algorithm, such that no two queens attack each other on an 8×8 chessboard.

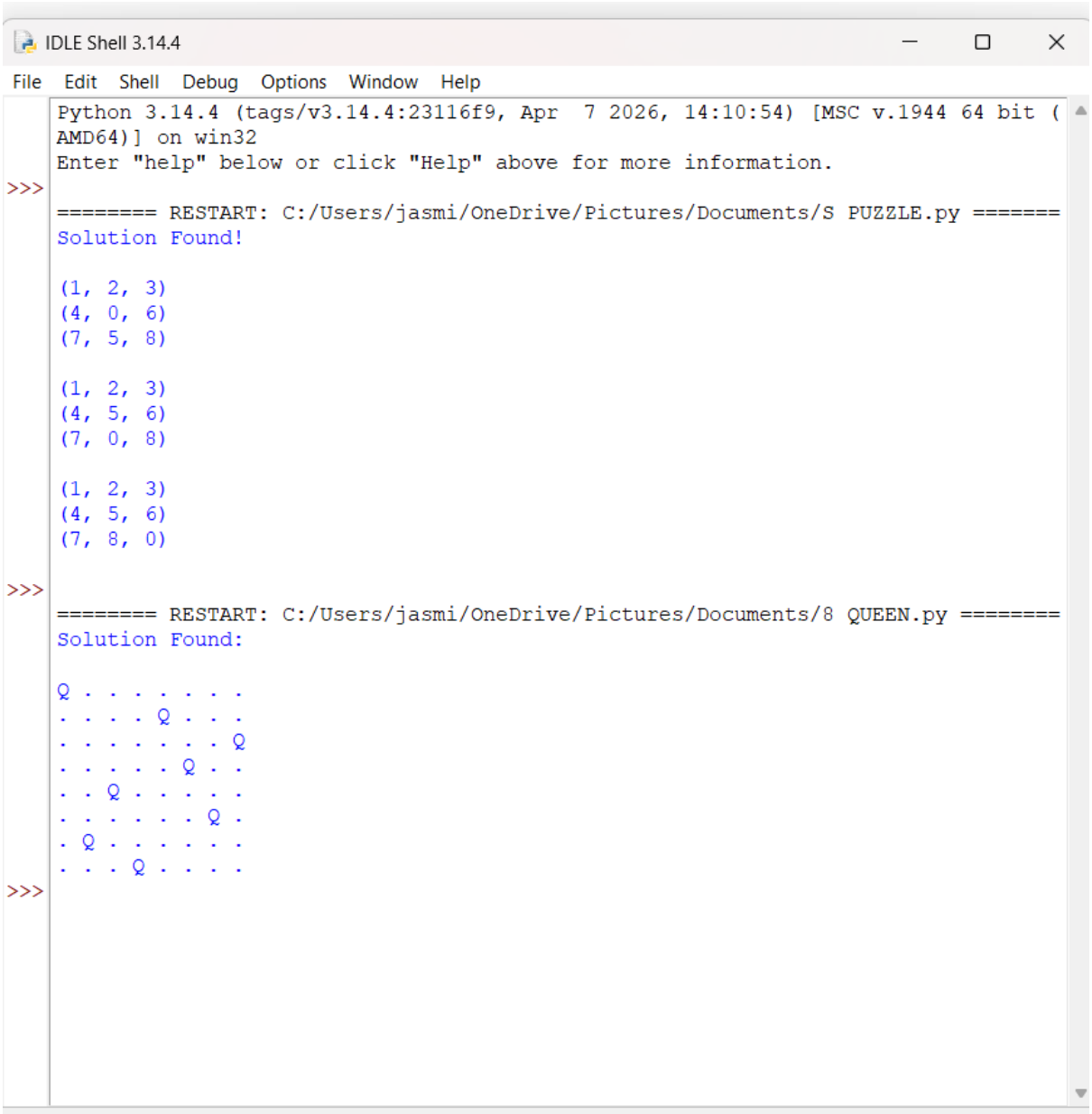
Algorithm

1. Create an empty 8×8 chessboard.
2. Start placing queens row by row.
3. Before placing a queen, check whether the position is safe:
    - No queen exists in the same column.
    - No queen exists in the left diagonal.
    - No queen exists in the right diagonal.
4. If the position is safe, place the queen.
5. Recursively place queens in the next row.
6. If no safe position is found, backtrack by removing the previously placed queen.
7. Continue until all 8 queens are successfully placed.
8. Display the final chessboard configuration.

8 QUEEN.py - C:/Users/jasmi/OneDrive/Pictures/Documents/8 QUEEN.py (3.14.4)

File Edit Format Run Options Window Help

```
# N-Queens Problem using Backtracking

N = 8

# Print the board
def print_board(board):
    for row in board:
        for cell in row:
            if cell:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

# Check if placing queen is safe
def is_safe(board, row, col):

    # Check column
    for i in range(row):
        if board[i][col]:
            return False

    # Check upper-left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < N:
        if board[i][j]:
            return False
        i -= 1
        j += 1

    return True

# Backtracking function
def solve(board, row):
    if row == N:
        return True

    for col in range(N):
        if is_safe(board, row, col):
```



RESULT:

Thus, the 8-Queen Problem was solved successfully using Python.

Program 3: Water Jug Problem using Python

Aim

To solve the Water Jug Problem using state-space search.

Algorithm

1. Start.
2. Initialize the capacities of Jug 1, Jug 2, and the target amount.
3. Create the initial state (0, 0).
4. Create an empty queue and insert the initial state.
5. Create an empty set to store visited states.
6. While the queue is not empty:
    - Remove the front state from the queue.
    - If the state is already visited, continue to the next state.
    - Mark the current state as visited.
    - If either jug contains the target amount, print the solution path and stop.
    - Generate the following successor states:
        - Fill Jug 1.
        - Fill Jug 2.
        - Empty Jug 1.
        - Empty Jug 2.
        - Pour water from Jug 1 to Jug 2.
        - Pour water from Jug 2 to Jug 1.

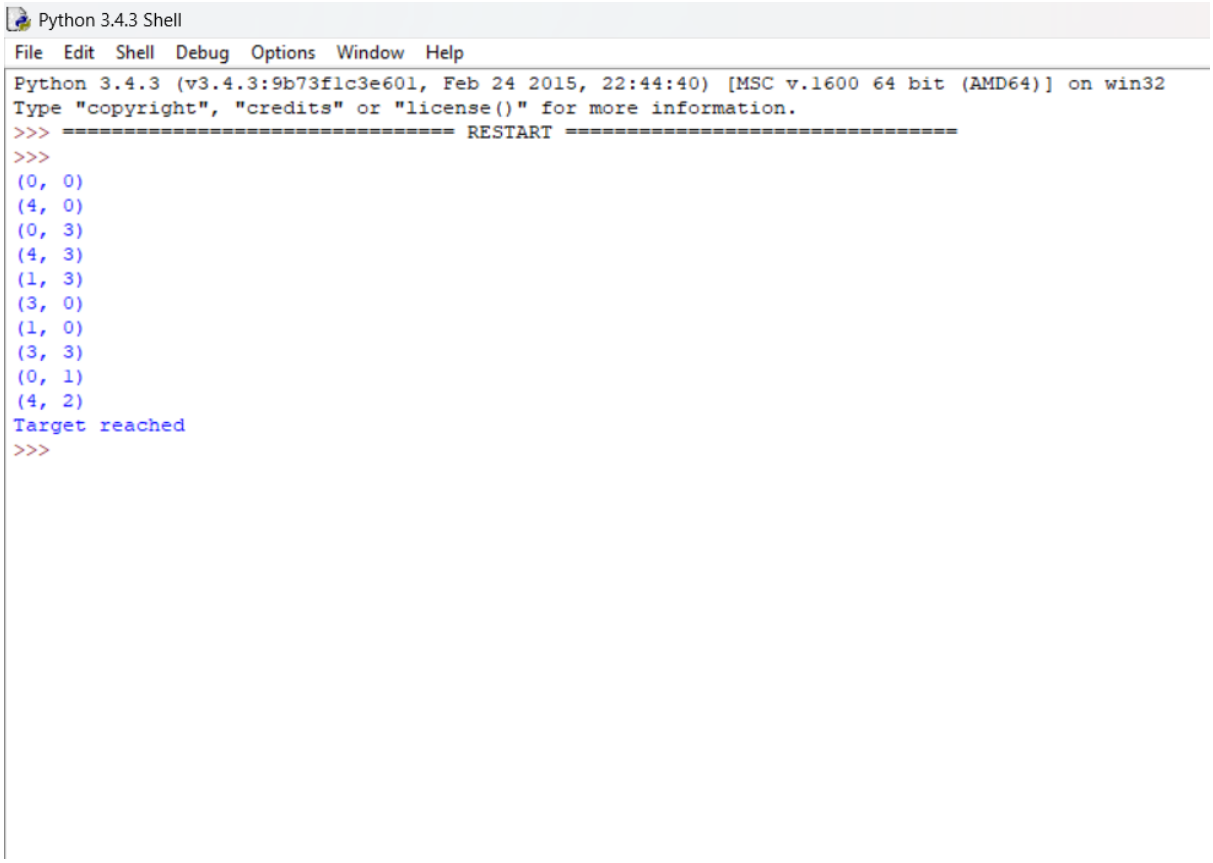
- Add all unvisited successor states to the queue.

7. If the queue becomes empty, print **"No Solution"**.
8. Stop.

water.py - C:/Users/jasmi/OneDrive/Desktop/water.py (3.4.3)

File Edit Format Run Options Window Help

```
from collections import deque

def water_jug(jug1, jug2, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        x, y = queue.popleft()

        if (x, y) in visited:
            continue
        visited.add((x, y))

        print((x, y))

        if x == target or y == target:
            print("Target reached")
            return

        next_states = [
            (jug1, y),
            (x, jug2),
            (0, y),
            (x, 0),
            (max(0, x-(jug2-y)), min(jug2, y+x)),
            (min(jug1, x+y), max(0, y-(jug1-x)))
        ]

        for state in next_states:
            if state not in visited:
                queue.append(state)

water_jug(4, 3, 2)
```



Result :

Thus, the Water Jug Problem was solved successfully using Python.

Program 4: Missionaries and Cannibals Problem

Aim:

To solve the Missionaries and Cannibals problem using State Space Search in Python.

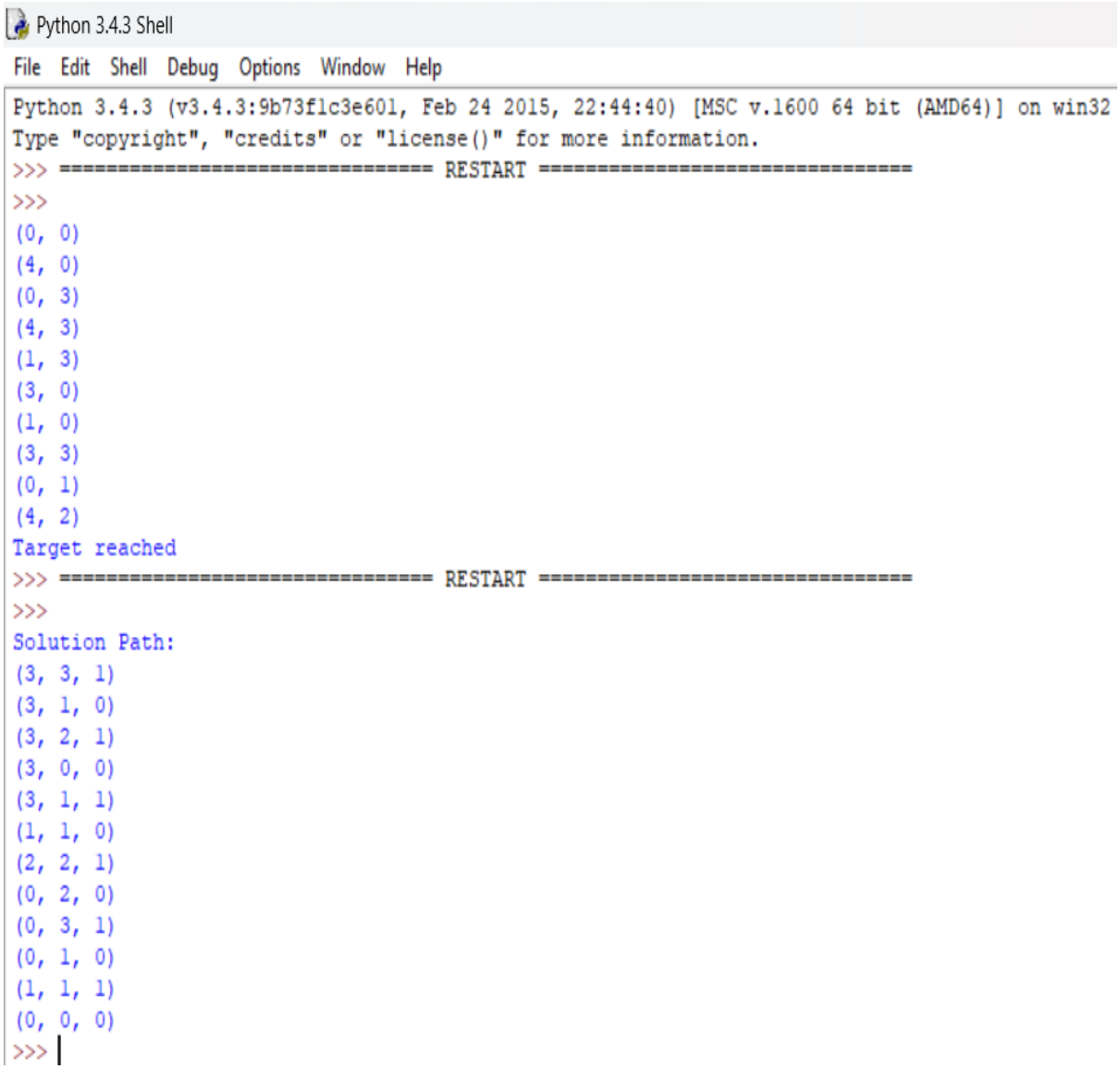
Algorithm:

1. Start with the initial state (3, 3, Left Bank).
2. Create a queue and insert the initial state.
3. Create a set to store visited states.
4. Remove the front state from the queue.
5. If it is the goal state (0, 0, Right Bank), print the solution path and stop.
6. Generate all possible boat moves:
    - Two missionaries
    - Two cannibals
    - One missionary
    - One cannibal
    - One missionary and one cannibal
7. For each generated state:
    - Check whether the state is valid (missionaries are never outnumbered by cannibals on either bank).
    - If valid and not visited, add it to the queue.
8. Repeat Steps 4–7 until the goal state is reached.
9. Display the sequence of states from the initial state to the goal state.

Missionaries and Cannibals Problem.py - C:/Users/jasmi/OneDrive/Desktop/Missionaries and Ca

File Edit Format Run Options Window Help

```
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False

    if (m_left > 0 and m_left < c_left):
        return False

    if (m_right > 0 and m_right < c_right):
        return False

    return True

def missionaries_cannibals():
    start = (3, 3, 1)
    goal = (0, 0, 0)

    queue = deque([(start, [])])
    visited = set()

    while queue:
        (m, c, boat), path = queue.popleft()

        if (m, c, boat) in visited:
            continue

        visited.add((m, c, boat))
        path = path + [(m, c, boat)]

        if (m, c, boat) == goal:
            print("Solution Path:")
            for state in path:
                print(state)
            return

        moves = [(2,0),(0,2),(1,0),(0,1),(1,1)]

        for dm, dc in moves:
            if boat == 1:
                new = (m-dm, c-dc, 0)
            else:
                new = (m+dm, c+dc, 1)

            if is_valid(new[0], new[1]):
                queue.append((new, path))

missionaries_cannibals()
```



Result

Thus, the Missionaries and Cannibals Problem was successfully solved using State Space Search (Breadth-First Search) in Python.

Program 5: Vacuum Cleaner Problem using Python

Aim

To implement the Vacuum Cleaner Problem using Python and clean all dirty rooms.

Algorithm

1. Start.
2. Initialize the environment with rooms and their states (Clean or Dirty).
3. Set the vacuum cleaner's initial position.
4. Check the current room.
5. If the current room is dirty, clean it.
6. Move the vacuum cleaner to the next room.
7. Repeat Steps 4–6 until all rooms have been visited.
8. Display the final state of all rooms.
9. Stop.

vaccum cleaner.py - C:/Users/jasmi/OneDrive/Desktop/vaccum cleaner.py (

File Edit Format Run Options Window Help

```
def vacuum_cleaner(rooms):
    for room in rooms:
        print(f"Checking Room {room}")

        if rooms[room] == "Dirty":
            print(f"Room {room} is Dirty.")
            print("Cleaning...")
            rooms[room] = "Clean"
        else:
            print(f"Room {room} is already Clean.")

        print()

    print("Final State of Rooms:")
    for room in rooms:
        print(f"Room {room}: {rooms[room]}")

# Initial State
rooms = {
    "A": "Dirty",
    "B": "Clean",
    "C": "Dirty",
    "D": "Dirty"
}

vacuum_cleaner(rooms)
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:1
Enter "help" below or click "Help" above for more info
>>>
========== RESTART: C:/Users/jasmi/OneDrive/Desktop/va
Checking Room A
Room A is Dirty.
Cleaning...

Checking Room B
Room B is already Clean.

Checking Room C
Room C is Dirty.
Cleaning...

Checking Room D
Room D is Dirty.
Cleaning...

Final State of Rooms:
Room A: Clean
Room B: Clean
Room C: Clean
Room D: Clean
>>>
```

Result

Thus, the Vacuum Cleaner Problem was successfully using the python.